МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Межинститутская базовая кафедра
Институт информационных технологий и кибербезопасности

Отчёт к лабораторной работе №5

Тема: **«Unit-тестирование»**

Дисциплина: **«Тестирование и отладка программного обеспечения»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки:
09.03.04 «Программная
инженерия»

Астраков Борис Александрович

_____

Проверил:

Ассистент МИБК

Потапов Иван Романович

_____

Ставрополь 2026

Цель работы: для разработанного приложения (пп. 1.4) продемонстрировать применение технологии unit-тестирования.

Ход работы:

Вариант проекта – игра «Собиратель монет» на pygame.

1. Цель тестирования

Проверить корректность изолированной работы основных компонентов игры: фабрик игровых объектов, сущностей, уровня, мультиплеерной логики, адаптера ввода и прокси звука.

2. Анализ требований

На основании тестируемого кода были выделены следующие проверяемые требования:

- фабрика должна создавать объекты нужных типов: Player, Coin, Hazard;
- клоны прототипов должны иметь независимые позиции и состояние;
- конструктор уровня должен отклонять объекты за границами уровня;
- игрок должен двигаться и оставаться в пределах игрового поля;
- урон игроку должен применяться с учётом периода неуязвимости;
- стили монет должны быть разделяемыми flyweight-объектами;
- бонусная монета должна декорировать значение и делегировать обновление базовой монете;
- двухпользовательский уровень должен содержать двух игроков и бонусные монеты;
- матч должен считать собранные монеты, очки и определять победителя;
- игроки в композитной модели должны двигаться независимо;
- адаптер ввода должен разделять управление стрелками и WASD;
- прокси звука не должен проигрывать звук при отключённом звуке.

3. Подход к тестированию

Использован модульный подход на базе unittest. Для изоляции внешних зависимостей применялись unittest.mock.Mock, в частности для проверки поведения SoundEffectsProxy. Тесты не требуют запуска графического окна и проверяют логику компонентов напрямую.

Покрытые техники и паттерны:

- проверка типов создаваемых объектов;
- проверка граничных условий и ограничений игрового поля;
- проверка состояния и изменения координат;
- проверка кулдауна неуязвимости;
- проверка разделяемых ресурсов / flyweight;
- проверка декоратора;
- проверка композитного поведения;
- проверка адаптера ввода;
- проверка прокси с помощью mock-объекта.

4. Состав тестов

Ниже приведён полный листинг unit-тестов приложения.

```
import unittest
from unittest.mock import Mock

import pygame

from coin_collector.audio import SoundEffectsProxy
from coin_collector.entities import BonusCoin, Coin, Hazard, Player
from coin_collector.factory import ArcadeObjectFactory,
CoinStyleFactory, ObjectKind
from coin_collector.input_adapter import InputFrame,
PygameInputAdapter
from coin_collector.level import GameLevelBuilder,
build_two_player_level
from coin_collector.multiplayer import TwoPlayerMatch


class EntityTests(unittest.TestCase):
    def test_factory_creates_requested_entities(self):
        factory = ArcadeObjectFactory()
        self.assertIsInstance(factory.create(ObjectKind.PLAYER, (40,
40)), Player)
        self.assertIsInstance(factory.create(ObjectKind.COIN, (50,
50)), Coin)
```

```
        self.assertIsInstance(factory.create(ObjectKind.HAZARD, (60,
60)), Hazard)

    def
test_prototype_clones_have_independent_positions_and_state(self):
        factory = ArcadeObjectFactory()
        first = factory.create(ObjectKind.COIN, (50, 50))
        second = factory.create(ObjectKind.COIN, (90, 90))
        self.assertIsInstance(first, Coin)
        self.assertIsInstance(second, Coin)
        first.rect.center = (10, 10)
        first.animation_time = 3
        self.assertEqual(second.rect.center, (90, 90))
        self.assertEqual(second.animation_time, 0)

    def test_level_builder_rejects_out_of_bounds_objects(self):
        builder = GameLevelBuilder(ArcadeObjectFactory(), (960, 640))
        builder.add_coin((0, 0))
        builder.add_hazard((0, 0))
        with self.assertRaises(ValueError):
            builder.build()

    def test_player_moves_and_stays_within_bounds(self):
        player = Player((50, 50))
        bounds = pygame.Rect(0, 0, 100, 100)
        player.update(InputFrame(movement=(1, 0)), 0.1, bounds)
        self.assertGreater(player.rect.centerx, 50)
        player.update(InputFrame(movement=(-1, -1)), 2.0, bounds)
        self.assertTrue(bounds.contains(player.rect))

    def test_player_damage_obeys_invulnerability_cooldown(self):
        player = Player((50, 50))
        self.assertTrue(player.take_damage(1.0))
        self.assertFalse(player.take_damage(1.0))
        self.assertEqual(player.health, 2)

    def test_coin_styles_are_shared_flyweights(self):
        factory = ArcadeObjectFactory()
        first = factory.create(ObjectKind.COIN, (50, 50))
        second = factory.create(ObjectKind.COIN, (90, 90))
        self.assertIs(first.style, second.style)
        self.assertIs(first.style, CoinStyleFactory.get("gold"))

    def test_bonus_coin_decorates_value_and_delegates_updates(self):
        coin = Coin((50, 50))
        bonus = BonusCoin(coin)
        bonus.update(0.5)
        self.assertEqual(bonus.value, 20)
        self.assertEqual(coin.animation_time, 0.5)
        self.assertIs(bonus.rect, coin.rect)


class MultiplayerTests(unittest.TestCase):
    def
test_two_player_level_builds_two_players_and_bonus_coins(self):
        level = build_two_player_level(ArcadeObjectFactory(), (960,
640))
```

```
        self.assertEqual(len(level.players), 2)
        self.assertEqual(len(level.coins), 11)
        self.assertEqual(sum(coin.value for coin in level.coins), 120)
        self.assertNotEqual(level.players[0].color,
level.players[1].color)

    def test_match_awards_coins_and_selects_the_higher_score(self):
        level = (
            GameLevelBuilder(ArcadeObjectFactory(), (960, 640))
            .set_player_start((200, 200))
            .add_player_start((700, 200))
            .build()
        )
        level.coins = [
            Coin((200, 200)),
            Coin((200, 200)),
            BonusCoin(Coin((700, 200))),
        ]
        match = TwoPlayerMatch(level)

        collected = match.update(InputFrame(), InputFrame(), 0.016)

        self.assertEqual(collected, 3)
        self.assertEqual(match.coin_counts, [2, 1])
        self.assertEqual(match.scores, [20, 20])
        self.assertTrue(match.finished)
        self.assertEqual(match.result_message, "Игрок 1 победил!")
        self.assertEqual(match.update(InputFrame(), InputFrame(),
0.016), 0)

    def test_player_composite_moves_players_independently(self):
        level = (
            GameLevelBuilder(ArcadeObjectFactory(), (960, 640))
            .set_player_start((200, 200))
            .add_player_start((700, 200))
            .build()
        )
        match = TwoPlayerMatch(level)

        match.update(
            InputFrame(movement=(1, 0)),
            InputFrame(movement=(-1, 0)),
            0.1,
        )

        self.assertGreater(level.players[0].rect.centerx, 200)
        self.assertLess(level.players[1].rect.centerx, 700)

    def test_input_adapter_separates_arrow_and_wasd_movement(self):
        keys = {key: False for key in (
            pygame.K_RIGHT,
            pygame.K_LEFT,
            pygame.K_DOWN,
            pygame.K_UP,
            pygame.K_d,
            pygame.K_a,
            pygame.K_s,
            pygame.K_w,
```

```
        )}
        keys[pygame.K_RIGHT] = True
        keys[pygame.K_w] = True

        frame = PygameInputAdapter().read([], keys)

        self.assertEqual(frame.player_one_movement, (1, 0))
        self.assertEqual(frame.player_two_movement, (0, -1))

    def test_sound_proxy_does_not_play_when_disabled(self):
        sound = Mock()
        proxy = SoundEffectsProxy(sound, lambda: False)

        proxy.play("coin")

        sound.play.assert_not_called()

    def test_sound_proxy_forwards_when_enabled(self):
        sound = Mock()
        proxy = SoundEffectsProxy(sound, lambda: True)

        proxy.play("coin")

        sound.play.assert_called_once_with("coin")


if __name__ == "__main__":
    unittest.main()
```

5. Результаты тестирования

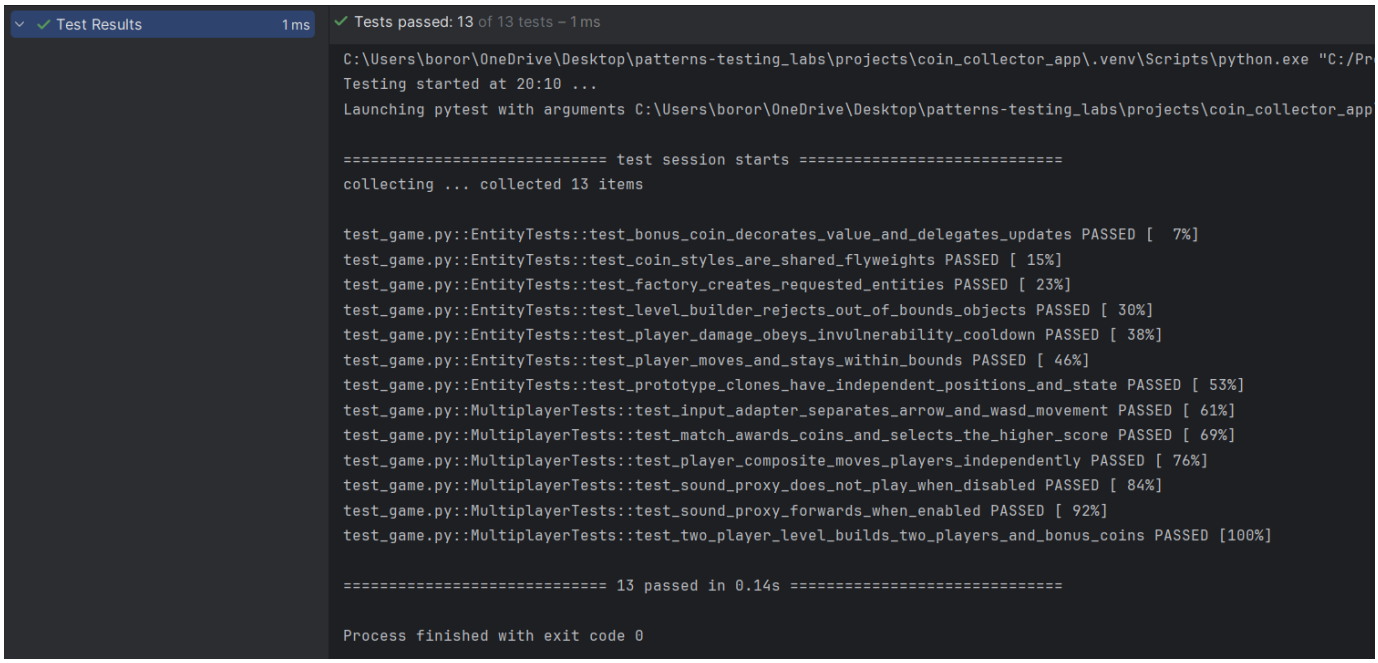


Рисунок 1. Вывод результатов тестов в PyCharm

Вывод:

Проделав работу, применил технологию unit-тестирования для разработанного приложения.